



1차 프로젝트_DATA/AI

NumPy 기반 영화 평점 분석 프로젝트

유정현 | 오현민 | 김태훈





TABLE OF CONTENTS

01 프로젝트 개요



02 프로젝트 구성: Step 1



03 프로젝트 구성: Step 2



04 프로젝트 구성: Step 3



05 프로젝트 구성: Step 4



06 프로젝트 구성: Step 5



07 프로젝트 구성: 프로젝트 확장



08 느낀 점 및 회고



프로젝트 개요



CHAPTER 01 🔍

프로젝트 개요



프로젝트 개요

IMDB 영화 데이터를 활용하여 NumPy를 이용한 데이터 분석을 수행합니다. 주어진 데이터에서 영화 평점과 장르별 특성을 분석하고, 연도별 평점 변화를 탐색하는 것이 목표입니다.



프로젝트 목표

IMDb 영화 데이터를 활용하여 평점 분석 및 데이터 처리 실습



사용 기술 / 도구

python : 전체적인 언어 사용
numpy : 데이터 처리
pandas : csv 파일 불러오기
re : 정규 표현식을 이용한 텍스트 전처리

프로젝트 구성



Step 1-1 : 데이터 불러오기

```
import numpy as np
import pandas as pd
import re
import matplotlib.pyplot as plt
|   # css 파일 읽기
df = pd.read_csv("IMDB top 1000.csv", encoding='utf-8')
|   # numpy 로 변환
data = df.to_numpy()

title = data[:,1]
|   # css 파일 읽기
df = pd.read_csv("IMDB top 1000.csv", encoding='utf-8')
|   # numpy 로 변환
data = df.to_numpy()
```

KEY POINTS

코드 해설: STEP 1-1

- 파일 불러오기에 필요한 내용 확인
 - a. import 를 통해 패키지를 불러옵니다.
 - b. pandas 를 통해 파일 읽기를 했습니다.
 - c. pandas 파일을 Numpy 배열로 전환하여 최종 Numpy 작업대기로 전환했습니다.



CHAPTER 02 🔍

Step 1-2 : 데이터 전처리

```

#title_genre_rate 컬럼 + title 년도 제거
title_genre_rate = data[:, [1, 4, 5]]
cleaned_titles = np.array([re.sub(r'\s*\(\d{4}\)', '', title) for title in title])
title_genre_rate[:, 0] = cleaned_titles
title_genre_rate[:, 0] = np.array([re.sub(r'^\d+\.', '', t) for t in title_genre_rate[:, 0]])

#released years 컬럼
released_years = np.array([title[-5:-1] for title in title])

# 최종 데이터 결합
final_data = np.column_stack((title_genre_rate, released_years))

# 데이터 크기(shape) 확인
print(final_data)
print(final_data.shape)

```

✓ 0.0s

KEY POINTS

코드해설: STEP 1-2

- 전체 예제문을 보고 전처리의 필요성 확인하기
 - a. 영화제목에 제작년도가 같이있는걸 확인하여 제작년도를 따로 추출하기
 - b. 정규표현식 re.sub를 이용하여 영화제목에 있는 제작년도를 추출하여 컬럼만 들기
 - 정규표현식 re.sub 를 이용하여 영화제목 앞쪽 번호 삭제하기
 - 영화제목에서 제작년도를 추출한 컬럼을 새로운 제작년도 변수에 생성하기
 - c. 예제문에서 필요한 데이터 분석준비를 마무리하고, 데이터크기를 확인



CHAPTER 02 🔍

Step 2 : 기본적인 데이터 탐색

```
# Step2
print("전체 영화 개수: ", final_data.shape[0])
print("평점 평균: ", np.mean(final_data[:, 2].astype(float)))
print("최고 평점: ", np.max(final_data[:, 2].astype(float)))
print("최저 평점: ", np.min(final_data[:, 2].astype(float)))
```

✓ 0.0s

전체 영화 개수: 1000
평점 평균: 8.0975
최고 평점: 9.3
최저 평점: 8.0

KEY POINTS

코드 해설: STEP 2

1. 전체 영화 개수를 출력

- `.shape[0]`을 이용하여 전체 행(row)의 개수를 나타내기

2. 전체 영화의 평균 평점, 최고 평점, 최저 평점 계산 출력

- 평균평점: `np.mean`을 이용하여 조건함수에 평점인덱스를 가져와서 이때, 타입을 `float`으로 바꿔서 나타냅니다.
- `max` 및 `min` 도 위 내용과 같이 이용하지만 `max`는 `np.max` 또는 `min` 이용한다.



CHAPTER 02 🔍

Step 3 : 평점이 높은 영화 찾기

```
# Step3
for i in range(final_data.shape[0]):
    if final_data[i, 2] == np.max(final_data[:, 2].astype(float)):
        print("최고 평점 영화 제목:\n", final_data[i, 0], "- 평점 :", final_data[i, 2])
```

✓ 0.0s

최고 평점 영화 제목:

The Shawshank Redemption - 평점 : 9.3

KEY POINTS

코드 해설: STEP 3

- 데이터에서 최고 평점(최대값) 찾고 목록으로 출력하기
 - a. 전체 데이터를 하나씩 대입을 해서 확인하기 위해 for문으로 대입하기
 - b. range를 이용하여 final_data.shape[0] 인덱스를 각 하나씩 대입하기
 - c. 이때, 조건문 if를 이용해서 전체 평점과 최대값 평점을 비교하기
 - d. 최종 최대값 평점에 대한 영화 제목과 평점을 나타내기



CHAPTER 02 🔍

Step 4 : 특정 장르별 평균 평점 분석

```
# step4
genre_1 = {}
for i in title_genre_rate:
    gen = i[1].split(',')
    for j in gen:
        j = j.strip()
        genre_1.setdefault(j, []).append(float(i[2]))

genre_rateavg = {}
genre_rate = []
# 장르만 추출
genre = list(genre_1.keys())
# 평점만 추출
for i in genre:
    genre_1[i] = np.mean(genre_1[i])
    genre_rate.append(round(float(genre_1[i]), 2))
    # 장르와 평점으로 딕셔너리 생성
for i in range(len(genre)):
    genre_rateavg[genre[i]] = genre_rate[i]
    # 평점 오름차순 정렬
def get_value(item):
    return item[1]
sorted_genre_rate = dict(sorted(genre_rateavg.items(), key = get_value, reverse = True))
for i in sorted_genre_rate:
    print(f"장르:{i}, 평점:{sorted_genre_rate[i]}")
```

KEY POINTS

코드 해설: STEP 4

1. 영화의 장르(Genre) 컬럼을 기반으로 모든 장르를 추출하기
2. 각 장르별로 포함된 영화들의 평균 평점을 계산
3. 장르별 평균 평점을 높은 순으로 정렬하여 출력



Step 4 : 특정 장르별 평균 평점 분석

```
# step4
genre_1 = {}
for i in title_genre_rate:
    gen = i[1].split(',')
    for j in gen:
        j = j.strip()
        genre_1.setdefault(j, []).append(float(i[2]))

genre_rateavg = {}
genre_rate = []
# 장르만 추출
genre = list(genre_1.keys())
```

KEY POINTS

코드 해설: STEP 4-1

1. 영화의 장르(Genre) 컬럼을 기반으로 모든 장르를 추출하기

- 영화 제목, 장르, 평점을 가지고 있는 변수를 반복문을 통해서 gen이란 변수에 넣을 때, 1번 인덱스만 넣습니다. 거기서 제거하여 변수에 하나씩 넣고 이때 다시 반복문을 전 작업에 넣었던 변수를 가지고 와서 앞뒤 공백을 제거하여 딕셔너리에서 장르와 평점을 넣고 장르만 list로 추출



CHAPTER 02 🔍

Step 4 : 특정 장르별 평균 평점 분석

```
# 장르만 추출
genre = list(genre_1.keys())
# 평점만 추출
for i in genre:
    genre_1[i] = np.mean(genre_1[i])
    genre_rate.append(round(float(genre_1[i]),2))
```

KEY POINTS

코드 해설: STEP 4-2

2. 각 장르별로 포함된 영화들의 평균 평점을 계산

- 장르만 추출하여 list를 만들었다면 각 장르별로 평점의 평균을 계산하여 추출
- 반복문을 이용하여 평균에 대한 값을 추출하여 list에 반환



Step4 : 특정 장르별 평균 평점 분석

KEY POINTS

코드 해설: STEP 4-3

1. 영화의 장르(Genre) 컬럼을 기반으로 모든 장르를 추출하기
2. 각 장르별로 포함된 영화들의 평균 평점을 계산
3. 장르별 평균 평점을 높은 순으로 정렬하여 출력

```
# 장르와 평점으로 딕셔너리 생성
for i in range(len(genre)):
    genre_rateavg[genre[i]] = genre_rate[i]
# 평점 오름차순 정렬
def get_value(item):
    return item[1]
sorted_genre_rate = dict(sorted(genre_rateavg.items(), key = get_value, reverse = True))
for i in sorted_genre_rate:
    print(f"장르:{i}, 평점:{sorted_genre_rate[i]}")
```



CHAPTER 02 🔍

Step 5 : 연도별 평점 변화 분석

```
# Step5
years1 = {}
for i in range (len(released_years)):
    years1.setdefault(int(released_years[i]), []).append(final_data[i, 2])
# print(years1)
years_rateavg = {}
years_rate = []
# 연도만 추출
years = list(years1.keys())
# 평점만 추출해서 평균 계산
for i in years:
    years1[i] = np.mean(years1[i])
    years_rate.append(round(float(years1[i]), 2))
```

KEY POINTS

코드 해설: STEP 5-1

1. 중복을 제거하고 제작년도와 평점만 추출해서 평균 계산

- setdefault로 중복된 연도를 빼고 연도와 평점을 갖고 있는 데이터로 변경
- 연도와 평점 데이터를 분리하고 평균 평점 계산



CHAPTER 02 🔍

Step 5 : 연도별 평점 변화 분석

```
# 연도와 평균 평점으로 딕셔너리 생성
for i in range(len(years)):
    years_rateavg[years[i]] = years_rate[i]
# 연도 오름차순 정렬
def get_keys(item):
    return item[0]
sorted_years_rate = dict(sorted(years_rateavg.items(), key = get_keys, reverse = False))
# 연도와 평균 평점 한 줄씩 출력
for i in sorted_years_rate:
    print(i, sorted_years_rate[i])
```

KEY POINTS

코드 해설: STEP 5-2

2. 연도와 평균 평점을 합치고 오름차순으로 정렬

- 연도와 평균 평점을 딕셔너리로 합쳐주었고 연도를 기준으로 오름차순으로 정렬하여 출력



CHAPTER 02 🔍

Step 5 : 연도별 평점 변화 분석

1920 8.1
1921 8.3
1924 8.2
1925 8.2
1926 8.1
1927 8.12
1928 8.1
1931 8.4
1934 8.1
1936 8.5
1937 8.1
1939 8.1
1940 8.12
1941 8.3
1942 8.35
1944 8.3
1946 8.6
1948 8.13
1949 8.1
1950 8.27
1951 8.1
1952 8.3
1953 8.11
1954 8.19
1955 8.2
...
2017 8.07
2018 8.08
2019 8.12
2020 8.7

KEY POINTS

코드 해설 STEP 5-2

2. 연도와 평균 평점을 합치고 오름차순으로 정렬

- 연도와 평균 평점을 딕셔너리로 합쳐주었고 연도를 기준으로 오름차순으로 정렬하여 출력

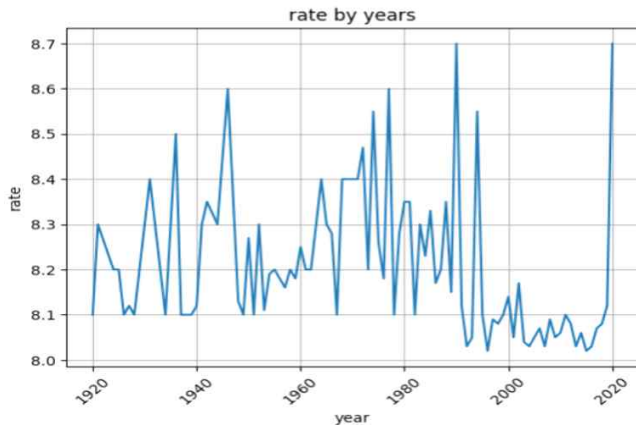


CHAPTER 02 🔍

Step 6 : 결과 시각화

```
# Step6
x = list(sorted_years_rate.keys())
y = list(sorted_years_rate.values())
plt.title("rate by years")
plt.xlabel("year")
plt.ylabel("rate")
plt.plot(x, y)
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```

0.1s



KEY POINTS

코드 해설 STEP 6

- **Matplotlib**를 사용하여 시각화
 - 가로축을 연도, 세로축을 평균 평점으로 설정
 - 보조선을 추가해서 가독성 향상



CHAPTER 02 🔍

프로젝트 확장 1 : 평점 8.0 이상 & 2000년대 이후 제작 영화 필터링

```
# 프로젝트 확장
# 특정조건 (예 평점 8.0 이상 & 2000년대 이후 제작) 충족 하는 영화만 필터링
movie_df = []
for i in final_data:
    if float(i[2]) >= 8.0 and int(i[3]) >= 2000:
        movie_df.append(i)
final_movie = np.array(movie_df)
print("평점 8.0 이상 & 2000년 이후 제작된 영화 개수: ", final_movie.shape[0])
print(f"평점 8.0 이상 & 2000년 이후 제작 영화:")
movie_years = final_movie[:, 3].astype(int)
year_idx = np.argsort(movie_years)[::-1] # 내림차순 정렬
final_movie = final_movie[year_idx]
# 제작년도, 제목, 평점 출력
for movie in final_movie[:5]: # 상위 5개 영화 출력
    movie_list = list(movie)
    print(f"제작년도:{movie_list[3]}, 제목:{movie_list[0]},평점:{movie_list[2]}")
```

평점 8.0 이상 & 2000년 이후 제작된 영화 개수: 573

평점 8.0 이상 & 2000년 이후 제작 영화:

제작년도:2020, 제목:Hamilton,평점:8.7

제작년도:2019, 제목:Parasite,평점:8.6

제작년도:2019, 제목:Marriage Story,평점:8.0

제작년도:2019, 제목:1917,평점:8.3

제작년도:2019, 제목:Avengers: Endgame,평점:8.4

KEY POINTS

코드 해설: 프로젝트확장 1

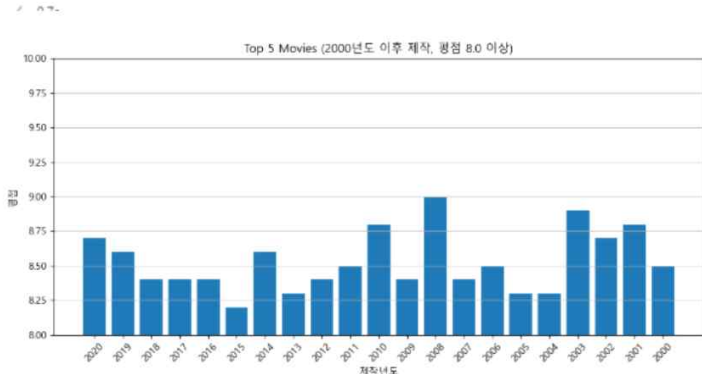
- 반복문과 조건문을 이용하고 평점과 제작년도를 비교할수있는 비교연산자 까지 이용하기
- 빈리스트를 만들고 반복문 for문으로 전처리가 완료가 된 데이터를 대입하여 가져옵니다. 이때, 가져오는 데이터의 인덱스를 지정하는데 여기서 필요한 건 제작년도, 영화제목, 평점 입니다. 따라서 필요한 인덱스중 영화제목 제외한 평점과 제작년도에 대한 비교연산자 수식으로 조건을 잡아서 빈리스트 양식에 반환합니다.
- 빈리스트를 반환 받아 최종적으로 Numpy배열로 전환을 합니다.
- 제작된 영화 개수를 구하라는 내용은 없었지만 영화의 개수가 너무많아 몇 개인지 확인하기 위해서 전체 개수를 나타냈습니다.
- 이후 내림차순 정렬을 제작년도로 잡고 argsort를 이용하여 정렬합니다.
- 이후 제작년도 제목 평점 출력을 상위 5개 영화 목록으로 마련해 최종적으로 나타냅니다.



CHAPTER 02 🔍

프로젝트 확장 1 : 평점 8.0 이상 & 2000년대 이후 제작 영화 필터링

```
# 그래프 그리기
plt.rcParams['font.family'] = 'Malgun Gothic' # 한글 폰트 설정
plt.rcParams['figure.figsize'] = (10, 5) # 그래프 크기 설정
plt.ylim(8.0, 10.0) # y축 범위 설정
plt.title("Top 5 Movies (2000년도 이후 제작, 평점 8.0 이상)")
plt.xlabel("제작년도")
plt.ylabel("평점")
plt.bar(final_movie[:, 3], final_movie[:, 2])
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```



KEY POINTS

데이터 시각화

- 그래프를 생성하여 조금 더 쉬운 데이터의 확인을 위해 나타냈습니다

CHAPTER 02 🔍

프로젝트 확장 2 : 특정 감독별 평균 평점 출력

```
# 프로젝트 확장
# 특정 감독별 평균 평점 계산
data1 = data[:, [8, 5]] # 평점과 감독 및 배우 컬럼
director_list = []
rate_list = []

for director, rate in zip(data[:,8], data[:,5]):
    director1 = director.split('|')[0].strip() # 감독 이름 추출
    director_list.append(director1.replace('Director:', '').strip())
    rate_list.append(float(rate))
print(director_list[:5]) # 감독 이름 5개 출력

director_name = "Christopher Nolan" # 특정 감독 이름

# 특정 감독의 평점
director_rate = [
    score
    for dir_name, score in zip(director_list, rate_list)
    if dir_name == director_name
]

# 평균 평점 계산

if director_rate:
    avg = sum(director_rate) / len(director_rate)
    print(f"{director_name} 감독의 평균 평점: {avg:.1f}")
else:
    print(f"{director_name} 감독의 영화가 없습니다.")
```

✓ 0.0s

```
['Frank Darabont', 'Francis Ford Coppola', 'Christopher Nolan', 'Francis Ford Coppola', 'Peter Jackson']
Christopher Nolan 감독의 평균 평점: 8.6
```

KEY POINTS

코드 해설: 프로젝트 확장 2

1. 감독과 배우 나누기 ('|')
2. 특정 감독을 지정하여 평점을 계산하여 출력

#키워드를 입력하세요

CHAPTER 02 🔍

프로젝트 확장 2 : 특정 감독별 평균 평점 출력

```
# 프로젝트 확장
# 특정 감독별 평균 평점 계산
data1 = data[:, [8, 5]] # 평점과 감독 및 배우 컬럼
director_list = []
rate_list = []

for director, rate in zip(data[:,8], data[:,5]):
    director1 = director.split('|')[0].strip() # 감독 이름 추출
    director_list.append(director1.replace('Director:', '').strip())
    rate_list.append(float(rate))
print(director_list[:5]) # 감독 이름 5개 출력
```

```
['Frank Darabont', 'Francis Ford Coppola', 'Christopher Nolan', 'Francis Ford Coppola', 'Peter Jackson']
```

KEY POINTS

코드 해설: 프로젝트 확장 2-1

- 우선 감독과 배우가 | 이걸로 나눈어진건데 파이프라고 이야기를 한다.
- 우선 빈리스트를 두개를 만들고 기존데이터를 가져와도되지만 여기서 오류가있다. data1을 가지고 만들어야했는데 zip을 이용해서 기존 데이터에서 가져오는 바람에 data1을 만들고서 사용을 못했다.
- 우선 zip을 통해 동시에 감독과 평점을 각 변수에 담는다.
- 이때 배우와 분리를 하기위해서 (|)를 중앙으로 놓고 앞쪽은 0인덱스 뒤쪽은 1인덱스로 하여 감독이름만 추출하기 위해서 0인덱스 기준으로 한다 그리고 앞뒤 공백을 제거하고 변수에 넣기
- 이때 감독이름만 있는게 아닌 감독이라는 컬럼명도 같이 있어서 이부분을 리스트에 추가하기전에 제거하고 이어서 다시 공백을 제거하여 최종적으로 리스트에 넣는다.
- 평점은 소수점으로 변환하여 리스트에 넣으면된다.
- 감독의 이름이 많이 나오는관계로 감독이름을 5개만 출력했다.

CHAPTER 02 🔍

프로젝트 확장 2 : 특정 감독별 평균 평점 출력

KEY POINTS

코드 해설: 프로젝트 확장 2-2

```

director_name = "Christopher Nolan" # 특정 감독 이름

# 특정 감독의 평점
director_rate = [
    score
    for dir_name, score in zip(director_list, rate_list)
    if dir_name == director_name
]

# 평균 평점 계산

if director_rate:
    avg = sum(director_rate) / len(director_rate)
    print(f"{director_name} 감독의 평균 평점: {avg:.1f}")
else:
    print(f"{director_name} 감독의 영화가 없습니다.")

```

Christopher Nolan 감독의 평균 평점: 8.6

- 우선 감독과 배우가 |이걸로 나누어진건데 파이프라고 이야기를 한다.
- 우선 빈리스트를 두개를 만들고 기존데이터를 가져와도되지만 여기서 오류가있다. data1을 가지고 만들어야했는데 zip을 이용해서 기존 데이터에서 가져오는 바람에 data1을 만들고서 사용을 못했다.
- 우선 zip을 통해 동시에 감독과 평점을 각 변수에 담는다.
- 이때 배우와 분리를 하기위해서 (")를 중앙으로 놓고 앞쪽은 0인덱스 뒤쪽은 1인덱스로 하여 감독이름만 추출하기 위해서 0인덱스 기준으로 한다 그리고 앞뒤 공백을 제거하고 변수에 넣기
- 이때 감독이름만 있는게 아닌 감독이라는 컬럼명도 같이 있어서 이부분을 리스트에 추가하기전에 제거하고 이어서 다시 공백을 제거하여 최종적으로 리스트에 넣는다.
- 평점은 소수점으로 변환하여 리스트에 넣으면된다.
- 감독의 이름이 많이 나오는관계로 감독이름을 5개만 출력했다.



CHAPTER 02 🔍

프로젝트 확장 3 : 특정 배우별 평균 평점 출력

```

# 프로젝트 확장
# 특정 배우별 평균 평점 계산
data1 = data[:, [8, 5]] # 평점과 감독 및 배우 컬럼
stars_list = []
rate_list = []

for star, rate in zip(data[:,8], data[:,5]):
    star1 = star.split('|')[1].strip() # 배우 이름 추출
    star1 = star1.replace('Stars:', '').strip() # 'Stars:' 제거
    star2 = [i.strip() for i in star1.split(',')] # 배우 이름 리스트 변환
    for j in star2:
        stars_list.append(j)
        rate_list.append(float(rate))
print(stars_list[:5]) # 배우 이름 5개 출력력

star_name = "William Sadler" # 특정 배우 이름

# 특정 배우의 평점
star_rate = [
    score
    for str_name, score in zip(stars_list, rate_list)
    if str_name == star_name
]

# 평균 평점 계산

if star_rate:
    avg = sum(star_rate) / len(star_rate)
    print(f"{star_name} 배우의 평균 평점: {avg:.1f}")
else:
    print(f"{star_name} 배우의 영화가 없습니다.")

```

KEY POINTS

코드 해설: 프로젝트 확장 3

- 위 특정 감독별과 마찬가지로 배우를 작성하면되는데 여기서 배우 이름은 한번 더 튜플로 묶인상태입니다. 그래서 배우이름을 한번더 반복문을 이용하여 리스트로 변환하는 작업을 하면 프로젝트확장2와 같은 작업이 완료가 됩니다.

#치위드를 입력

['Tim Robbins', 'Morgan Freeman', 'Bob Gunton', 'William Sadler', 'Marlon Brando']
 William Sadler 배우의 평균 평점: 9.0



CHAPTER 08 🔍

느낀 점 및 회고

어려웠던 점

-오현민

numpy 데이터를 다루는 방법이 어색했고 numpy의 다양한 내장함수들을 사용하기 어려웠습니다. 그 중에서 데이터의 전처리 방법이 가장 어려웠습니다.

-유정현

공부한 문법의 실전 적용 방법이 익숙하지 않아, 강의 자료를 반복하여 복습하면서 했습니다.

-김태훈

함수의 기본은 어느정도 익혀 알겠지만 실제적으로 쓰임을 할때에는 아직까진 어려움을 느껴 한계를 느꼈습니다.

배운 점

-오현민

numpy 데이터를 다루는 다양한 함수와 csv 파일의 데이터 전처리 방법, 다양한 문법의 사용을 배울 수 있었습니다. github, vs code 등 협업 프로젝트에서 어떤 프로그램이 쓰이고 git의 쓰임을 이해할 수 있었습니다.

-유정현

프로젝트를 통해 문법을 직접 응용해보면서 개념들을 더욱 깊게 이해할 수 있었습니다.

-김태훈

강의자료를 통한 방법들이 나와 비교적 강의내용을 다시 보고 작업은 하였던 것 같습니다. 이제는 어느 부분에서 어떤 조건, 반복에 대한 내용을 조금은 알 수 있었던 계기가 되었습니다.



느낀 점 및 회고

다음에 개선하고 싶은 부분

-오현민

익숙하지 않은 문법이 많아 gpt의 도움을 많이 받았는데 gpt의 도움을 최소한으로 받고 프로젝트를 마무리하고 싶습니다. github로 프로젝트를 받아서 해결을 해보고 싶습니다.

-유정현

처음 진행한 프로젝트였기 때문에, 보고서 작성 방식이나 발표 형식에 대한 구체적인 안내가 추후에 있었으면 좋겠습니다.

-김태훈

예제 글에 대한 내용을 보고 아직까진 아 이걸 어떤걸 이용하면 되겠구나!? 라는 내용 파악이 안되는걸 개선하고 싶습니다. 그럴때마다 토론하듯이 GPT를 이용을 하였는데, 결국엔 알게 되었을 때 허무함이 조금 많이 컸던것같아 이 부분은 조금 더 풀어줄 강의자료 또는 내용으로 배움을 받아야할 것 같습니다.

감사합니다